Single-Cell RNA-seq Analysis Pipeline

Detailed Methodology, Results Tables, Step-by-Step Interpretation, and Final Analysis Report

Notebook link: https://github.com/Rameensajjad/scRNA-Analysis

Report Component	Details
Analysis type	Human PBMC single-cell RNA-seq analysis
Sequencing platform	10x Genomics Chromium V3 chemistry
Preprocessing tool	STARsolo on Galaxy
Downstream tools	AnnData, Scanpy, Scrublet, Leiden clustering, CellTypist
Main output	Clean AnnData object, clusters, cell type annotations, marker genes, and result tables
Prepared for	Assignment / laboratory analysis report

Note: This report is written as a descriptive analysis report. It explains the purpose, method, expected output, result interpretation, and limitations of each step rather than presenting raw code only.

Executive Summary

This report describes a complete end-to-end single-cell RNA sequencing (scRNA-seq) analysis workflow performed on human peripheral blood mononuclear cell (PBMC) data. The workflow begins with raw FASTQ preprocessing using STARsolo on Galaxy and continues into Python-based downstream analysis using AnnData and Scanpy. The purpose of the analysis is to convert raw sequencing reads into an interpretable cellular map that separates immune cell populations, assigns biological identities to clusters, and identifies marker genes that explain each cluster.

The analysis includes quality assessment, alignment, barcode correction, UMI counting, cell calling, quality control filtering, doublet removal, normalization, highly variable gene selection, dimensionality reduction, Leiden clustering, cell type annotation, and differential expression testing. Each step improves the reliability of the data and reduces technical noise so that the final clusters represent real biological variation rather than sequencing artifacts.

Table 1. Overall workflow summary

Major Stage	Main Objective	Main Result
Raw preprocessing	Convert FASTQ reads into a gene-barcode matrix	Filtered MEX matrix from STARsolo
Quality control	Remove empty droplets, damaged cells, and poor-quality genes	Cleaned cell-gene matrix
Normalization	Correct differences in sequencing depth	Comparable expression values across cells
Feature selection	Identify most informative genes	Top 2,000 highly variable genes
Dimensionality reduction	Summarize complex expression patterns	PCA and UMAP representations
Clustering	Group transcriptionally similar cells	Leiden clusters at multiple resolutions
Annotation	Assign biological identity to clusters	Immune cell type labels
Differential expression	Identify marker genes per cluster	Cluster-specific gene signatures

1. Background and Introduction

1.1 What is Single-Cell RNA Sequencing?

Single-cell RNA sequencing is a transcriptomic technology that measures gene expression separately in thousands of individual cells. In conventional bulk RNA-seq, RNA is extracted from a mixture of cells and the final signal represents an average expression profile. This averaging can hide important biological differences between cell types. In contrast, scRNA-seq preserves cell-level information and allows researchers to detect rare populations, compare immune cell states, identify developmental trajectories, and understand heterogeneity within a sample.

In this analysis, the sample type is human PBMCs. PBMCs contain several immune cell types, including T cells, B cells, natural killer cells, monocytes, dendritic cells, and sometimes erythroid-lineage cells or plasma cells. Because PBMCs are heterogeneous, they are a suitable training dataset for learning scRNA-seq analysis and cell type annotation.

1.2 Aim of the Analysis

- To preprocess raw 10x Genomics sequencing reads and generate a filtered expression matrix.
- To understand how single-cell data are stored and organized using the AnnData object.

- To perform downstream Scanpy analysis, including QC, normalization, clustering, annotation, and marker gene detection.
- To interpret final immune cell populations using marker genes and automated annotation methods.

1.3 Why Each Step is Necessary

Raw single-cell sequencing data contain several technical artifacts. These include empty droplets, barcode sequencing errors, PCR duplicates, ambient RNA, low-quality cells, and doublets. If these problems are not removed, downstream analysis may create false clusters or misidentify cell types. Therefore, the workflow uses multiple quality control and computational correction steps before interpreting the biology.

Table 2. Common scRNA-seq problems and the analysis step used to handle them

Problem in Raw Data	Why it Matters	Step Used to Address It
Empty droplets	They contain ambient RNA and can look like weak cells	Cell calling and minimum gene filtering
Barcode errors	Wrong barcodes create artificial cells	Whitelist matching and barcode correction
PCR duplicates	Duplicates inflate gene counts	UMI deduplication
Damaged cells	They show high mitochondrial expression	Mitochondrial QC filtering
Doublets	Two cells captured together appear as a fake hybrid cell	Scrublet doublet detection
Sequencing depth variation	Cells with more reads may appear falsely more active	Normalization
High-dimensional noise	Thousands of genes make patterns difficult to see	HVG selection, PCA, and UMAP

2. Dataset Description

The analysis uses human PBMC data generated using 10x Genomics Chromium V3 chemistry. In the raw preprocessing part, FASTQ files are processed on Galaxy using STARsolo. In the Python downstream part, two sample matrices are loaded and combined for analysis. The dataset is appropriate for learning because PBMCs contain well-known immune populations with established marker genes.

Table 3. Dataset and input information

Attribute	Description
Sample type	Human PBMCs from healthy donor material
Sequencing chemistry	10x Genomics Chromium V3
Raw input	FASTQ files containing cell barcodes, UMIs, and cDNA reads
Read structure	Read 1 includes cell barcode and UMI; Read 2 contains transcript sequence
Reference genome	hg19 / GRCh37
Gene annotation	Homo_sapiens.GRCh37.75.gtf
Barcode whitelist	3M-february-2018 whitelist for 10x V3 barcodes
Galaxy training cell count	Approximately 272 to 300 cells after cell calling/QC
Python downstream input	Two filtered feature matrices from samples s1d1 and s1d3
Approximate loaded cells	About 11,000 cells before downstream filtering

3. Part 1 - Raw Data Preprocessing Using STARsolo on Galaxy

3.1 Purpose of Raw Preprocessing

The purpose of raw preprocessing is to convert sequencing reads into a structured gene expression matrix. A 10x Genomics experiment produces FASTQ files, but downstream single-cell tools require a matrix showing how many UMIs were detected for each gene in each cell. STARsolo performs this conversion by extracting barcodes and UMIs, aligning transcript reads to the genome, assigning reads to genes, collapsing PCR duplicates, and producing a sparse matrix.

3.2 Step 1: Raw Read Quality Assessment

Before alignment, the FASTQ files are examined using FastQC and summarized using MultiQC. This step checks whether the sequencing reads are of acceptable quality and whether there are signs of contamination, adapter sequence, or severe base quality decline. In scRNA-seq, some duplication is expected because highly expressed transcripts are repeatedly sequenced, but extremely poor quality or excessive adapter contamination can reduce alignment performance.

Table 4. Raw read quality assessment metrics

QC Metric	What It Measures	Acceptable / Expected Pattern	Interpretation
Per-base sequence quality	Quality score at each position of the read	Mean Phred score generally above 30	High scores indicate reliable base calling
Per-sequence quality	Distribution of average read quality	Most reads should have good quality	Low-quality tails may affect alignment
GC content	Base composition across reads	Human reads often around moderate GC range	Large deviations may indicate contamination or bias
Sequence duplication	Repeated read sequences	Moderate to high duplication is common in scRNA-seq	Very extreme duplication may reflect low complexity
Adapter content	Presence of sequencing adapter sequences	Should be low after trimming/preprocessing	High adapter content can reduce mapping

3.3 Step 2: Barcode and UMI Extraction

In 10x Chromium V3 data, Read 1 stores the cell barcode and UMI. The cell barcode identifies the droplet/cell of origin, while the UMI labels the original mRNA molecule. STARsolo extracts the barcode and UMI from Read 1 and uses Read 2 as the transcript read for alignment. Barcodes are compared with the 10x whitelist. A whitelist is a list of known valid barcodes used by the chemistry. Allowing one mismatch helps correct minor sequencing errors without accepting highly unreliable barcodes.

Table 5. Barcode and UMI extraction settings

Element	Length / Setting	Role in Analysis
Cell barcode	16 bp	Identifies the cell or droplet of origin
UMI	12 bp	Identifies unique original mRNA molecules
Barcode whitelist	10x V3 whitelist	Validates expected barcode sequences
Mismatch tolerance	1 mismatch	Corrects likely single-base sequencing errors
Discarded barcodes	2 or more mismatches	Removed because they are likely invalid or noisy

3.4 Step 3: Alignment to the Reference Genome

After barcode and UMI extraction, transcript reads are aligned to the human reference genome. The reference genome and gene annotation guide the software in determining where each read originated and which gene it should be assigned to. In this analysis, hg19/GRCh37 and Ensembl GRCh37.75 gene annotation were used. Alignment quality is critical because poor alignment can lead to incorrect gene counts or loss of reads.

Table 6. STARsolo alignment and counting parameters

STARsolo Parameter	Value Used	Reason
Reference genome	hg19 / GRCh37	Human genome reference used for alignment
Gene annotation	GRCh37.75.gtf	Defines gene coordinates and exon structures
Chemistry	10x Chromium V3	Ensures correct barcode and UMI structure
Barcode length	16 bp	Matches 10x V3 barcode design
UMI length	12 bp	Matches 10x V3 UMI design
Junction overhang	100 bp	Supports splice-aware alignment
Barcode mismatch	1	Corrects minor barcode sequencing errors

3.5 Step 4: Gene Assignment and UMI Deduplication

Once reads are aligned, STARsolo assigns them to genes using the gene annotation file. However, sequencing reads do not directly equal mRNA molecules because PCR amplification can create many duplicate reads from one molecule. The UMI system solves this problem. Reads that share the same cell barcode, gene assignment, and UMI are treated as duplicates from the same original molecule and collapsed into a single count. This produces a UMI count matrix, which is more biologically meaningful than raw read counts.

Table 7. Logic of UMI deduplication

Counting Situation	How STARsolo Treats It	Resulting Count
Same cell barcode + same gene + same UMI	PCR duplicates are collapsed	Counted as 1 molecule
Same cell barcode + same gene + different UMI	Different original molecules	Counted separately
Different cell barcode + same gene + same UMI	Different cells	Counted separately for each cell
Invalid barcode	Not confidently assigned to a cell	Removed or ignored

3.6 Step 5: Cell Calling Using Droplet-Based Statistics

Not every barcode represents a real cell. Some droplets are empty but still contain ambient RNA released from lysed cells. Cell calling separates real cells from empty droplets by ranking barcodes according to UMI count. Real cells usually have high UMI totals and form a plateau on a knee plot, while empty droplets form a long low-count tail. The threshold around the knee or inflection point is used to decide which barcodes are retained as cells.

Table 8. Interpretation of barcode rank/knee plot

Barcode Group	Typical UMI Count Pattern	Biological Meaning	Action
High-count barcode plateau	High UMI and gene counts	Likely real cells	Retain

Transition / knee region	Intermediate counts	Borderline barcodes	Assess statistically
Low-count tail	Very low UMI counts	Likely empty droplets or ambient RNA	Remove

3.7 STARsolo Output Files

The STARsolo output is compatible with downstream single-cell analysis tools. The most important output is the MEX matrix folder, consisting of a matrix file, barcode file, and feature file. Together these files represent the sparse gene-by-cell count matrix.

Table 9. STARsolo output files

Output File	Format	Description / Use
matrix.mtx	Matrix Market sparse matrix	Contains UMI counts for genes across barcodes
barcodes.tsv	Tab-separated text	Contains retained cell barcode identifiers
features.tsv	Tab-separated text	Contains gene IDs and gene names
Alignment log	Text file	Summarizes mapping statistics and alignment quality
BAM file	Binary alignment map	Stores aligned reads with cell barcode and UMI tags

3.8 Preprocessing Results

Table 10. Raw preprocessing result summary

Metric	Observed / Reported Value	Interpretation
Total barcodes detected	Approximately 5,200	Includes both real cells and background/empty droplet barcodes
Cells passing QC / cell calling	272 cells	High-confidence cells retained for analysis in Galaxy training workflow
Uniquely mapped reads	Greater than 70%	Good alignment quality and appropriate reference use
Final output format	Filtered MEX matrix	Ready for downstream analysis in Scanpy or similar tools

Interpretation: The preprocessing results indicate that the raw data were successfully converted into a filtered gene expression matrix. The difference between total detected barcodes and final retained cells is expected in droplet-based scRNA-seq because many droplets are empty or contain insufficient RNA. A uniquely mapped read percentage above 70% suggests that the reference genome and annotation were appropriate and that sequencing quality was acceptable for downstream analysis.

4. Part 2 - AnnData Data Structure

4.1 Purpose of AnnData

AnnData is the main data structure used by Scanpy and the scverse ecosystem. It stores the expression matrix and all associated annotations in a single object. This is important because single-cell analysis generates many linked data types: cell metadata, gene metadata, dimensionality reduction coordinates, graph matrices, clustering results, and plotting information. AnnData keeps these components aligned using shared cell and gene indices.

Table 11. Main components of the AnnData object

AnnData Field	Shape / Type	What It Stores	Example Use
adata.X	Cells x genes matrix	Active expression data	Used for normalization, PCA, and plotting
adata.obs	Cell metadata table	QC metrics, sample labels, clusters, annotations	Filter cells or compare samples
adata.var	Gene metadata table	Gene IDs, mitochondrial flags, HVG status	Select genes and marker features
adata.layers	Dictionary of matrices	Alternative expression matrices	Store raw counts and normalized values
adata.obsm	Cell x dimension arrays	PCA, UMAP, t-SNE coordinates	Visualization and embedding
adata.obsp	Cell x cell matrices	Neighbor graph and distances	Leiden clustering and UMAP
adata.uns	Dictionary	Unstructured results and parameters	Stores clustering settings and color maps

4.2 Importance of Shared Indexing

The most important principle of AnnData is that every data component is aligned by cell and gene identifiers. Each row in the expression matrix corresponds to the same cell represented in adata.obs and adata.obsm. Each column corresponds to the same gene represented in adata.var. This design prevents accidental mismatch between expression values and metadata during filtering or subsetting.

Table 12. Shared indexing in AnnData

Index	Meaning	Why It Matters
obs_names	Unique cell barcodes	Ensures every row corresponds to the correct cell
var_names	Gene names or gene IDs	Ensures every column corresponds to the correct gene
Subsetting cells	Reduces rows across all relevant fields	Maintains consistent cell-level metadata
Subsetting genes	Reduces columns in X/layers and rows in var	Maintains correct gene annotations

5. Part 3 - Downstream Analysis Using Scanpy

5.1 Environment Setup and Data Loading

The downstream analysis was performed in Python using Scanpy and related packages. Two samples, s1d1 and s1d3, were loaded from filtered 10x H5 matrices and combined into a single AnnData object. Combining samples allows joint clustering and comparison, while storing the sample identity in the metadata allows the analysis to track batch or sample-specific effects.

Table 13. Python tools used in downstream analysis

Package / Tool	Purpose in Workflow
scanpy	Core preprocessing, QC, normalization, PCA, UMAP, clustering, and plotting
anndata	Stores the single-cell dataset and metadata
scrublet	Detects likely doublets
leidenalg and igraph	Required for Leiden graph clustering

celltypist	Automated immune cell annotation
pandas and numpy	Data manipulation and numerical operations
matplotlib	Figure generation and export

Table 14. Data loading and merging steps

Loading Step	Purpose	Output
Read 10x H5 files	Import filtered feature-barcode matrices	AnnData object for each sample
Make gene names unique	Avoid duplicate gene name errors	Clean var_names
Concatenate samples	Combine s1d1 and s1d3	Single AnnData object
Add sample label	Track origin of each cell	adata.obs sample column
Make cell names unique	Avoid duplicated barcodes across samples	Unique obs_names

5.2 Quality Control: Gene-Level Flags

Before filtering, specific gene categories were flagged because they provide information about cell quality and contamination. Mitochondrial genes indicate cell stress or membrane damage. Ribosomal genes can reflect transcriptional or technical variation. Hemoglobin genes may suggest erythrocyte or erythroid contamination. These gene categories are not automatically bad, but their proportions help identify problematic cells.

Table 15. Gene categories used for QC

Gene Category	Pattern Used	Why It Is Checked	Interpretation of High Percentage
Mitochondrial genes	Gene names starting with MT-	Proxy for cell stress or damage	Damaged, dying, or low-quality cells
Ribosomal genes	Gene names starting with RPS or RPL	Proxy for ribosomal RNA/protein expression	May indicate technical bias or biological state
Hemoglobin genes	HB genes excluding some special cases	Checks erythrocyte/erythroid contamination	Possible red blood cell contamination or erythroid cells

5.3 Quality Control: Cell and Gene Filtering

After calculating QC metrics, cells and genes were filtered. Cells with very few detected genes are likely empty droplets or severely damaged cells. Cells with extremely high counts or gene numbers may be doublets. Genes expressed in too few cells provide little information and increase noise. The workflow used minimum gene and minimum cell filters to remove the weakest observations and features.

Table 16. Cell quality control metrics and interpretation

QC Metric	Low Value Suggests	High Value Suggests	Typical Action
n_genes_by_counts	Empty droplet or dead cell	Potential doublet or multiplet	Remove extreme low and suspicious high cells
total_counts	Low RNA content or damaged cell	Potential doublet or high RNA content cell	Inspect distribution and filter outliers
pct_counts_mt	Usually healthy if moderate/low	Stressed or dying cell	Apply upper threshold

pct_counts_ribo	May reflect low cellular complexity	Possible ribosomal bias	Monitor or use as covariate
pct_counts_hb	Non-erythroid cells should be low	Erythrocyte contamination or erythroid lineage	Monitor and interpret biologically

Table 17. Applied filtering logic

Filtering Rule	Reason	Expected Effect
Minimum genes per cell: 100	Removes droplets/cells with very low transcript diversity	Improves cell quality
Minimum cells per gene: 3	Removes genes detected in too few cells	Reduces noise and computation
Doublet removal after Scrublet	Removes artificial mixed cell profiles	Improves cluster purity
Mitochondrial percentage inspection	Identifies damaged cells	Prevents stress-driven clusters

5.4 Quality Control Results

Table 18. Downstream QC result summary

Stage	Approximate Cell Number / Result	Interpretation
After loading two samples	About 11,000 cells	Combined dataset before strict downstream QC
After min_genes / min_cells filtering	More than 10,000 cells retained	Low-quality cells and rare uninformative genes removed
After doublet removal	Reduced according to predicted doublet rate	Likely artificial multiplets removed
Final clean dataset	Used for PCA, UMAP, clustering, annotation	Suitable for biological interpretation

Interpretation: The filtering strategy retained most cells from the combined dataset while removing weak or unreliable observations. This indicates that the dataset was broadly usable and did not suffer from severe initial quality issues. The exact final number depends on the doublet rate and any additional thresholds applied during visual inspection.

5.5 Doublet Detection Using Scrublet

Doublets are a major source of error in droplet-based scRNA-seq. They occur when two cells are captured in the same droplet and sequenced as if they were one cell. This creates artificial expression profiles that may combine markers from two cell types, for example a T cell and a monocyte. If not removed, doublets can appear as false clusters or confuse annotation.

Scrublet addresses this by simulating artificial doublets from the observed data and then comparing each real cell to these simulated profiles. Cells with high doublet scores are flagged as predicted doublets. Running Scrublet per sample or batch is preferred because doublet score distributions can vary between samples.

Table 19. Doublet detection outputs

Scrublet Output	Meaning	Use in Analysis
doublet_score	Continuous probability-like score	Higher values indicate stronger doublet-like profile
predicted_doublet	Boolean/categorical doublet call	Used to remove likely doublets

batch-wise scoring	Scores calculated by sample	Avoids comparing different sample distributions incorrectly

5.6 Normalization and Log Transformation

Cells can have different total read or UMI counts because of sequencing depth, capture efficiency, or cell size. Without normalization, cells with more reads may appear to have higher expression for many genes simply because they were sequenced more deeply. Normalization rescales counts so cells are more comparable. Log transformation then compresses large expression values and stabilizes variance, making patterns easier to detect.

Table 20. Normalization and transformation steps

Step	What Is Done	Why It Is Important
Preserve raw counts	Raw integer matrix saved in a layer	Allows later use of original counts
Normalize total counts	Each cell is scaled to a comparable total	Corrects sequencing depth differences
Log1p transformation	Applies log(1+x) to normalized values	Reduces dominance of highly expressed genes
Use normalized X	Downstream methods use transformed matrix	Improves PCA and clustering stability

5.7 Highly Variable Gene Selection

Single-cell datasets contain thousands of genes, but many are either not expressed or do not vary meaningfully across cells. Highly variable genes are genes whose expression varies more than expected and therefore carries information about cell identity or state. Selecting 2,000 HVGs reduces noise and computational burden while retaining the main biological signals needed for clustering.

Table 21. Highly variable gene selection summary

Feature Selection Decision	Value / Approach	Interpretation
Number of HVGs selected	2,000 genes	Common choice for PBMC-scale Scanpy workflows
Batch-aware selection	Sample label used during HVG calculation	Reduces sample-specific bias
Genes not selected	Low-information or stable genes	Not used for PCA-based structure detection
Main benefit	Focuses on informative expression variation	Improves clustering and visualization

5.8 Dimensionality Reduction: PCA, Neighbors, and UMAP

After HVG selection, dimensionality reduction is used to summarize thousands of gene expression measurements into a smaller number of informative dimensions. PCA identifies major axes of variation in the dataset. A k-nearest neighbor graph is then built in PCA space to represent relationships between similar cells. UMAP uses this graph to produce a two-dimensional visualization where similar cells are placed near each other.

Table 22. Dimensionality reduction and graph construction

Method	Input	Output	Purpose
PCA	Normalized expression of HVGs	Principal components	Captures major expression variation

kNN graph	PCA coordinates	Cell-cell neighbor graph	Defines local similarity between cells
UMAP	Neighbor graph	2D embedding	Visualizes cell populations and cluster structure
Leiden clustering	Neighbor graph	Cluster labels	Groups similar cells into communities

5.9 Leiden Clustering

Leiden clustering is a graph-based community detection method. It groups cells that are strongly connected in the neighbor graph. The resolution parameter controls how many clusters are produced. Low resolution creates broad lineage-level groups, while high resolution detects smaller subclusters that may represent rare cell types or cell states. In this analysis, multiple resolutions were tested to compare cluster granularity.

Table 23. Leiden clustering resolutions

Resolution	Expected Number of Clusters	Best Use	Interpretation
0.02	3-5 broad clusters	Major lineage overview	Useful for broad separation such as lymphoid vs myeloid
0.5	10-15 clusters	Cell type annotation	Balanced resolution for PBMC annotation
2.0	20-30 clusters	Rare subpopulation discovery	May split major types into fine states

Interpretation: Resolution 0.5 was the most suitable for main annotation because it provides enough clusters to separate common immune cell types without over-fragmenting the dataset. Very low resolution is useful for general lineage structure, while very high resolution should be interpreted carefully because it can split one cell type into multiple technical or state-based clusters.

5.10 Cell Type Annotation

Cell type annotation assigns biological names to computational clusters. This analysis used two complementary strategies: automated annotation using CellTypist and manual marker gene validation. Automated annotation compares expression profiles to reference immune cell models, while marker validation checks whether known genes are enriched in the expected clusters. Combining both approaches increases confidence.

Table 24. Cell type annotation methods

Annotation Method	How It Works	Strength	Limitation
CellTypist	Matches cells to a trained immune reference model	Fast and standardized	Depends on reference quality
Majority voting	Assigns cluster-level labels based on cell predictions	Reduces noisy single-cell labels	May hide rare mixed populations
Marker gene scoring	Scores known marker gene sets per cell/cluster	Biologically interpretable	Requires accurate marker dictionary
Manual validation	Compares markers with expected PBMC biology	Improves confidence	Requires expert judgement

Table 25. Marker genes used for cell type interpretation

Cell Type	Key Marker Genes	Biological Interpretation
CD14+ Monocytes	FCN1, CD14, LYZ, S100A8, S100A9	Classical inflammatory monocytes involved in innate immunity
CD16+ Monocytes	FCGR3A, TCF7L2, LYN, MS4A7	Non-classical/intermediate monocytes associated with surveillance and inflammation
CD4+ T cells	CD4, IL7R, CCR7, LEF1, TCF7	Helper and naive/memory T cell populations
CD8+ T cells	CD8A, CD8B, GZMK, GZMA, CCL5	Cytotoxic T cell lineage and effector memory signatures
NK cells	GNLY, NKG7, KLRB1, KLRD1	Innate cytotoxic lymphocytes
B cells	MS4A1, CD79A, PAX5, BLK, BCL11A	B lymphocytes involved in antigen recognition
Plasma cells	MZB1, JCHAIN, IGKC, XBP1, PRDM1	Antibody-secreting differentiated B cells
Dendritic cells	FCER1A, CLEC10A, CD1C, CLEC9A	Antigen-presenting immune cells
Erythroblasts	HBA1, HBB, MKI67, GYPA	Erythroid lineage or red-cell-associated signal
pDC	GZMB, IL3RA, COBLL1, TCF4	Plasmacytoid dendritic cell-like immune population

5.11 Differential Expression Analysis

Differential expression analysis identifies genes that are enriched in one cluster compared with other clusters. In this workflow, the Wilcoxon rank-sum test was used because scRNA-seq data are sparse, zero-inflated, and generally not normally distributed. After ranking marker genes, a fold-change filter was applied to retain genes with stronger biological effect sizes.

Table 26. Differential expression workflow

Analysis Step	Purpose	Interpretation
Rank genes by cluster	Find genes most enriched in each Leiden cluster	Produces candidate marker genes
Wilcoxon test	Non-parametric comparison of expression distributions	Suitable for sparse single-cell data
Fold-change filtering	Retain markers with meaningful expression differences	Improves biological interpretability
Dendrogram construction	Compare similarity between clusters	Shows relationships among annotated groups
Dot plot visualization	Display marker expression across clusters	Supports annotation and reporting

Table 27. Example marker interpretation for Cluster 3 - monocyte signature

Gene	Function / Interpretation	Why It Supports Monocyte Identity
LYZ	Lysozyme involved in antimicrobial defense	Canonical monocyte and myeloid marker
ACTB	Beta-actin involved in cytoskeletal structure	Housekeeping/cytoskeletal signal; not specific alone
S100A6	Calcium-binding protein associated with inflammation	Supports immune activation signal
S100A4	Associated with cell migration and immune activation	Commonly seen in activated immune populations
CST3	Cystatin C, often expressed in myeloid cells	Supports myeloid/monocyte lineage

Interpretation: Cluster 3 shows a strong monocyte-like signature because it contains LYZ, S100-family genes, and CST3. LYZ is particularly useful because it is widely used as a myeloid/monocyte marker. ACTB alone is not a cell-type-specific marker, but the combined signature supports the annotation when interpreted with the other genes.

6. Complete Result Summary

Table 28. Complete result interpretation summary

Result Category	Main Finding	Interpretation
Preprocessing	Filtered matrix successfully generated	Raw FASTQ data were converted into usable count matrix
Cell calling	272 high-confidence cells in Galaxy workflow	Expected reduction from total barcodes due to empty droplets
Downstream dataset	Approximately 11,000 cells loaded across two samples	Larger combined dataset used for Python analysis
QC filtering	Low-quality cells and genes removed	Improved reliability of downstream clustering
Doublet detection	Predicted doublets removed	Reduced artificial hybrid cell profiles
HVG selection	2,000 variable genes selected	Focused analysis on informative genes
Clustering	Multiple Leiden resolutions tested	Resolution 0.5 most useful for cell type annotation
Cell annotation	Major immune populations identified	Clusters reflect meaningful PBMC biology
Differential expression	Cluster-specific markers detected	Marker genes validated biological labels

7. Biological Interpretation of the Final Analysis

The final analysis successfully identified major immune populations expected in PBMC data. Lymphoid populations included CD4+ T cells, CD8+ T cells, NK cells, and B cells. Myeloid populations included CD14+ monocytes, CD16+ monocytes, and dendritic cells. Plasma cells and erythroblast-like populations were also identified using specific marker genes. The presence of these groups indicates that the workflow captured real cellular heterogeneity rather than random technical variation.

The strongest evidence for successful analysis is the agreement between clustering, automated annotation, and marker gene expression. When clusters separate clearly in UMAP space and their marker genes match known immune biology, the annotation is considered reliable. For example, monocyte clusters show LYZ and S100 genes, B cell clusters show MS4A1 and CD79A, NK cells show GNLY and NKG7, and T cell clusters show CD4/CD8-related markers.

8. Limitations and Recommended Improvements

Table 29. Limitations and future improvements

Limitation	Impact on Analysis	Recommended Improvement
Small Galaxy training subset	Reduced statistical power in preprocessing demonstration	Process full dataset rather than training subset
chrX or limited genome training mode	Not representative of whole transcriptome	Use full human genome reference locally or on adequate compute
No ambient RNA correction	Ambient RNA may inflate expression in empty droplets or low-quality cells	Apply SoupX or CellBender

No explicit batch correction	Sample-specific effects may influence clustering	Apply Harmony, BBKNN, Scanorama, or scVI
hg19 reference genome	Older annotation may miss updated gene models	Use GRCh38/hg38 for modern analysis
Marker database limitations	Online databases unavailable in some environments	Use curated marker dictionary and manual validation
No trajectory analysis	Cell transitions or differentiation paths not studied	Apply Monocle3, scVelo, or PAGA if relevant
No independent validation	Annotation depends on computational evidence	Validate with flow cytometry or known reference datasets where possible

9. Final Conclusion

This report demonstrates a complete scRNA-seq analysis workflow from raw sequencing reads to biological interpretation. STARsolo preprocessing converted raw FASTQ files into a filtered gene expression matrix by extracting barcodes and UMIs, aligning reads, assigning genes, and collapsing PCR duplicates. The downstream Scanpy workflow then cleaned the dataset, removed predicted doublets, normalized expression values, selected highly variable genes, reduced dimensionality, performed Leiden clustering, annotated cell types, and identified cluster-specific marker genes.

The final results are consistent with expected PBMC biology. Major immune cell types, including T cells, B cells, NK cells, monocytes, dendritic cells, plasma cells, and erythroid-associated populations, were identified using a combination of automated annotation and marker gene validation. Differential expression analysis further supported cluster identities by identifying meaningful marker signatures such as LYZ, S100A4, S100A6, and CST3 for monocyte-like clusters.

Overall, the pipeline provides a reproducible and interpretable framework for single-cell transcriptomic analysis. With additional improvements such as batch correction, ambient RNA removal, and updated genome references, the analysis can be further strengthened for publication-level or research-grade applications.